

TRABAJO PRÁCTICO INTEGRADOR

PROGRAMACIÓN 2

JUNIO 2026

ALUMNO: AGUSTÍN CASSANO
COMISIÓN 2

Índice

1.	Marco Teórico	3
2.	Decisiones Técnicas y Arquitectura	4
3.	Sistema en funcionamiento	5
4.	Dificultades y Soluciones	11
5.	Enlaces del Proyecto	12
6.	Bibliografía y Webgrafía	13

Marco Teórico

El desarrollo del sistema Food Store se fundamenta exhaustivamente en los principios de la Programación Orientada a Objetos (POO), utilizando Java 21 como lenguaje principal. La solución emplea persistencia transaccional en memoria mediante colecciones, garantizando aislamiento de procesos durante la ejecución.

Los conceptos fundamentales aplicados abarcan todo el espectro de la POO:

- **Herencia y Clases Abstractas:** Se diseñó una superclase abstracta Base que centraliza los metadatos comunes (ID, estado de eliminación y timestamp de creación mediante LocalDateTime). Las entidades principales (Producto, Categoria, Usuario, Pedido, DetallePedido) heredan de esta estructura, reduciendo la duplicación de código y estandarizando la instanciación.
- **Polimorfismo e Interfaces:** Se implementó la interfaz Calculable, la cual define un comportamiento estandarizado (calcularTotal()). La clase Pedido implementa este contrato, asegurando que el total se recalcula dinámicamente cada vez que se alteran sus componentes.
- **Composición y Relaciones:** Se modelaron relaciones 1..N y N..1 respetando el dominio del negocio. Destaca la relación de composición fuerte entre Pedido y DetallePedido, donde el pedido es responsable absoluto del ciclo de vida de sus detalles (mediante métodos propios como addDetallePedido).
- **Tipado Fuerte mediante Enums:** Para garantizar la consistencia de los datos en estados críticos, se utilizaron enumeraciones para definir el Rol del usuario, el Estado del pedido y la FormaPago.
- **Estructuras de Datos Dinámicas (Colecciones):** La gestión del estado del sistema recae en listas dinámicas (ArrayList), permitiendo la indexación y modificación de registros en tiempo de ejecución.
- **Manejo de Excepciones Propias:** Se extendió la clase Exception para crear excepciones de negocio (EntidadNoEncontradaException, StockInvalidoException, ValidacionException), logrando un control de flujo robusto mediante bloques try-catch en la capa de presentación.

Decisiones Técnicas y Arquitectura

La arquitectura del sistema fue diseñada utilizando un patrón de separación lógica en tres capas, desacoplando el modelo de dominio de las reglas de negocio y de la interfaz de usuario:

1. Capa de Entidades (Modelo de Dominio / Paquete entities y enums) Esta capa es el núcleo del sistema. Las entidades son objetos inteligentes que no solo almacenan datos, sino que encapsulan lógica intrínseca. Por ejemplo, la clase `DetallePedido` calcula su propio subtotal automáticamente en su constructor. Las asociaciones bidireccionales, como la de Usuario y Pedido, se manejan protegiendo la integridad referencial (ej. el método `agregarPedido` dentro de Usuario asocia ambas partes simultáneamente).

2. Capa de Servicios (Reglas de Negocio / Paquete services) Actúa como intermediaria. Aloja las colecciones `ArrayList` y expone los métodos CRUD. Toda validación compleja se procesa aquí. Se diseñó para que reciba parámetros primitivos o de entidad, realice las comprobaciones (ej. validar si un email es único recorriendo la lista de usuarios) y, si detecta una anomalía, "lance" (throw) una excepción hacia la capa superior, protegiendo así el estado de las colecciones.

3. Capa de Presentación (Interfaz de Usuario / Clase Main) La interacción con el operador ocurre exclusivamente aquí a través de Scanner. Su responsabilidad se limita a mostrar menús, capturar el input, invocar a los servicios y capturar las excepciones para mostrar mensajes amigables, evitando que el programa finalice por errores de usuario.

Implementación de "Soft Delete" (Baja Lógica): Para mantener el historial de ventas y la auditoría, no se utiliza la eliminación física. El borrado de registros invierte el atributo booleano `eliminado = true` heredado de la clase Base. Los métodos de listado y búsqueda en los servicios excluyen automáticamente estos registros.

Sistema en funcionamiento

En este apartado se adjuntan capturas de pantalla evidenciando el funcionamiento del sistema.

Menú principal

```
run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: |
```

Submenus:

```
--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione:
```

```
--- GESTIÓN DE PRODUCTOS ---
1. Listar
2. Crear
3. Eliminar
0. Volver al menú principal
Seleccione: |
```

Creando una categoría:

```
--- GESTIÓN DE CATEGORÍAS ---  
1. Listar  
2. Crear  
3. Editar  
4. Eliminar  
0. Volver al menú principal  
Seleccione: 2  
Nombre de la categoría: Pizzas  
Descripción: Pizzas caseras con masa madre.  
Categoría creada con éxito. ID asignado: 1
```

Creando un producto:

```
--- GESTIÓN DE PRODUCTOS ---  
1. Listar  
2. Crear  
3. Eliminar  
0. Volver al menú principal  
Seleccione: 2  
Nombre del producto: Pizza roquefort  
Precio: 20000  
Descripción: Pizza casera con queso roquefort.  
Stock inicial: 20  
ID: 1 | Nombre: Pizzas | Descripción: Pizzas caseras con masa madre.  
ID de la Categoría: 1  
? Producto creado con éxito. ID asignado: 1
```

Nota: al crear un producto, el sistema pide que se ingrese a que categoría pertenece el mismo.

Creando un usuario

```
--- GESTIÓN DE USUARIOS ---  
1. Listar  
2. Crear  
0. Volver al menú principal  
Seleccione: 2  
Nombre: Agustin  
Apellido: Cassano  
Mail (único): agustin123@gmail.com  
Celular: 358545303  
Contraseña: contrasegural23  
Seleccione Rol (1. ADMIN | 2. USUARIO):  
1  
? Usuario creado con éxito. ID asignado: 1
```

Creando un pedido:

```
--- GESTIÓN DE PEDIDOS ---
1. Listar Pedidos
2. Crear Pedido (Asignar detalles)
0. Volver al menú principal
Seleccione: 2
ID: 1 | Nombre: Agustin Cassano | Mail: agustin123@gmail.com | Rol: ADMIN

Ingrese ID del Usuario para el pedido: 1
Forma de pago (1. EFECTIVO | 2. TARJETA | 3. TRANSFERENCIA):
1
ID: 1 | Nombre: Pizza roquefort | Precio: $20000,00 | Stock: 20 | Categoría: Pizzas
Ingrese ID del producto a agregar (0 para finalizar): 1
Cantidad: 3
Producto agregado al carrito.
ID: 1 | Nombre: Pizza roquefort | Precio: $20000,00 | Stock: 17 | Categoría: Pizzas
Ingrese ID del producto a agregar (0 para finalizar): 0
Pedido #1 registrado con éxito. Total: $60000.0
```

Nota: al crear pedido, el sistema pide que se asigne un Usuario existente para la creación del mismo. Si no se ingresa un usuario o caracter válido el sistema lanza un error deteniendo la creación del pedido, evitando así modificar con datos incorrectos o que se produzca la creación de pedidos "sin" usuario.

```
Ingrese ID del Usuario para el pedido:
Error: Formato numérico incorrecto.
```

Modificar:

El sistema permite modificar datos existentes que se quisieran cambiar. En este ejemplo se muestra como se cambia la descripción de una categoría. Como se puede observar, el programa permite dejar en blanco (en este caso el nombre de la categoría) si no se quiere modificar.

```
--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione: 3
ID: 1 | Nombre: Pizzas | Descripción: Pizzas caseras con masa madre.
ID de la categoría a editar: 1
Nuevo nombre (Dejar vacío para omitir):
Nueva descripción (Dejar vacío para omitir): Pizzas de elaboracion casera.
Categoría actualizada correctamente.
```

Listado: El sistema permite listar datos registrados desde los distintos submenus.

Desde Categorías:

```
--- GESTIÓN DE CATEGORÍAS ---  
1. Listar  
2. Crear  
3. Editar  
4. Eliminar  
0. Volver al menú principal  
Seleccione: 1  
ID: 1 | Nombre: Pizzas | Descripción: Pizzas de elaboracion casera.
```

Desde Productos:

```
--- GESTIÓN DE PRODUCTOS ---  
1. Listar  
2. Crear  
3. Eliminar  
0. Volver al menú principal  
Seleccione: 1  
ID: 1 | Nombre: Pizza roquefort | Precio: $20000,00 | Stock: 17 | Categoría: Pizzas
```

Desde Usuarios:

```
--- GESTIÓN DE USUARIOS ---  
1. Listar  
2. Crear  
0. Volver al menú principal  
Seleccione: 1  
ID: 1 | Nombre: Agustin Cassano | Mail: agustin123@gmail.com | Rol: ADMIN
```

Nota: Al listar datos de usuario el sistema no muestra datos sensibles, como la contraseña.

Desde Pedidos:

```
--- GESTIÓN DE PEDIDOS ---  
1. Listar Pedidos  
2. Crear Pedido (Asignar detalles)  
0. Volver al menú principal  
Seleccione: 1  
ID: 1 | Usuario: Agustin Cassano | Estado: PENDIENTE | Pago: EFECTIVO | Total: $60000,00 | Fecha: 2026-06-19
```


Demostración de Robustez y Manejo de Excepciones

En el siguiente apartado se expone el comportamiento del sistema ante escenarios de error y vulneración de reglas de negocio. A través de las siguientes capturas de pantalla, se demuestra la efectividad de las excepciones personalizadas creadas específicamente para este proyecto (tales como ValidacionException, StockInvalidoException y EntidadNoEncontradaException).

El objetivo de este diseño es garantizar la tolerancia a fallos y la protección de las colecciones en memoria. Como se evidencia a continuación, la correcta intercepción de estos eventos en la capa de presentación evita cierres abruptos de la aplicación (crashes), aborta transacciones inconsistentes antes de que afecten los datos globales y provee retroalimentación semántica y clara al operador.

NumberFormatException (Error de Tipo de Dato)

```
--- GESTION DE PRODUCTOS ---  
1. Listar  
2. Crear  
3. Eliminar  
0. Volver al menú principal  
Seleccione: 2  
Nombre del producto: Hamburguesas  
Precio: mil pesos  
Error de formato: asegúrese de ingresar números correctos (ej: precio 1500.50).
```

ValidacionException (Regla de Negocio: Mail Único)

```
--- GESTION DE USUARIOS ---  
1. Listar  
2. Crear  
0. Volver al menú principal  
Seleccione: 2  
Nombre: Carlos  
Apellido: Perez  
Mail (único): agustin123@gmail.com  
Celular: 5345345  
Contraseña: contra  
Seleccione Rol (1. ADMIN | 2. USUARIO):  
2  
Error: El mail 'agustin123@gmail.com' ya se encuentra registrado.
```

StockInvalidoException (Regla de Negocio Crítica)

```
--- GESTIÓN DE PEDIDOS ---
1. Listar Pedidos
2. Crear Pedido (Asignar detalles)
0. Volver al menú principal
Seleccione: 2
ID: 1 | Nombre: Agustin Cassano | Mail: agustin123@gmail.com | Rol: ADMIN

Ingrese ID del Usuario para el pedido: 1
Forma de pago (1. EFECTIVO | 2. TARJETA | 3. TRANSFERENCIA):
2
ID: 1 | Nombre: Pizza roquefort | Precio: $20000,00 | Stock: 17 | Categoría: Pizzas
Ingrese ID del producto a agregar (0 para finalizar): 1
Cantidad: 150
Error: Stock insuficiente para 'Pizza roquefort'. Stock actual: 17
Creación de pedido abortada para evitar inconsistencias.
```

EntidadNoEncontradaException (Búsqueda Fallida)

```
--- GESTIÓN DE CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione: 4
ID: 1 | Nombre: Pizzas | Descripción: Pizzas de elaboracion casera.
ID de la categoría a eliminar: 9
¿Está seguro? (S/N): s
Error: No se encontró una categoría activa con el ID 9
```

Dificultades y Soluciones

Durante el ciclo de desarrollo, se identificaron y resolvieron obstáculos técnicos vinculados al diseño orientado a objetos y la consistencia en memoria:

- Sincronización de Totales y Subtotales (Composición):
 - Dificultad: Asegurar que el total del Pedido estuviera siempre actualizado al agregar o remover un DetallePedido, sin exponer la colección interna de detalles a modificaciones externas.
 - Solución: Se encapsuló la colección de detalles. Se crearon métodos obligatorios en la entidad Pedido (addDetallePedido, deleteDetallePedidoByProducto). Al invocarse estos métodos, se autoejecuta la implementación de la interfaz Calculable (calcularTotal()), garantizando que la suma de subtotales refleje exactamente el contenido del carrito.
- Mantenimiento de la Integridad en Transacciones Fallidas:
 - Dificultad: Al crear un pedido complejo desde la consola, si el operador cargaba varios productos y el sistema fallaba en el último (por falta de stock), los productos previos ya habían sido descontados lógicamente, corrompiendo la colección.
 - Solución: Se diseñó un patrón de carga en pasos dentro de PedidoService. Primero se instancia un pedido "temporal". Los detalles y descuentos de stock se realizan sobre objetos validados. Solo si el ciclo se completa sin arrojar StockInvalidoException, el pedido se anexa a la colección general.
- Restitución de Inventario:
 - Dificultad: Aplicar la baja lógica a un Pedido mantenía los productos asociados retenidos.
 - Solución: Se extendió la lógica en la capa de servicios para que, al marcar un pedido como eliminado = true, un bucle itere sobre sus detalles y reincorpore las cantidades al stock de cada Producto original.

Enlaces del Proyecto

Repositorio de Código Fuente (GitHub):

Demostración Audiovisual (Video):

Bibliografía y Webgrafía

-Oracle Corporation. (2023). Java Platform, Standard Edition & Java Development Kit Version 21 API Specification. Recuperado de la documentación oficial.

-Eckel, B. (2006). Thinking in Java (4th ed.). Prentice Hall.

-Material de estudio, consignas de UML y lineamientos técnicos de la materia Programación 2, Tecnicatura Universitaria en Programación. Universidad Tecnológica Nacional (2026).